

Teach Yourself GraphQL in 2026

An Anti-blueprint

■ The shape of a query

- Operation type – `query` means "I want to read something"
- Arguments – filter or parameterize the field
- Selection set – the curly braces; describe what you want back
- Nesting – a flight has a price; a price has an amount and a currency

You ask for exactly what you need. The response comes back in exactly that shape.

That's basically the whole idea.

"Mostly. Though we'd call those 'legs' not 'stops.'"

```
"A stop is a layover. A leg is a segment –  
JFK to Heathrow is one leg even if you stop in Dublin.  
They're not the same thing in our system."
```



```
"You're not learning GraphQL.  
GraphQL is teaching you – about your own API."
```



```
"We've got a Postgres database for the core flight data,  
and then a third-party pricing API we call out to."
```


■ While Claude thinks... Nullability.

```
origin: String
```

nullable – might come back **null**

Most type systems: non-null by default, explicitly opt into nullable.

GraphQL: nullable by default. ■ opts into non-null.

```
origin: String!
```

non-null – will always have a value

schema arrives

`origin: String` → should be `String!` A flight without an origin doesn't make sense.

`price: Price` → stays nullable. Price comes from a third-party API. If that service is down, do we want the entire query to fail?


```
"Yeah, actually. We degrade gracefully  
when the pricing service is unavailable."
```


■ The resolver

This is the whole mechanism. Look at it once.

```
Query: {  
  flights: async (_, args) => {  
    const res = await fetch(`http://localhost:3001/flights`)  
    return res.json()  
  }  
}
```

■ "The resolver is just a function. It can call your database,
■ your REST API, your legacy service. GraphQL doesn't care."


```
"The booking confirmation. The booking ID, the status,  
and probably the flight details so you can show a summary screen."
```


■ While Claude thinks... Variables.

```
mutation BookFlight($flightId: ID!, $passengerId: ID!) {  
  bookFlight(flightId: $flightId, passengerId: $passengerId) {  
    booking {  
      id  
      status  
    }  
  }  
}
```

`$flightId: ID!` – variable name, type, non-null.

The operation declares what it needs. The client provides it at runtime. Clean separation.

mutation arrives

Look at the selection set on the flight inside the booking.

```
origin, destination, departureTime, legs -
```

We've seen those fields before. That's our dream query.

A fragment is a named, reusable selection set.

It belongs to a type – `on Flight` – not to an operation.

Change it in one place, it updates everywhere.

We'll come back to a more powerful use of fragments at the end.

OPERATION TYPES

Query Mutation — Subscription *

* real-time — out of scope today

ANATOMY

Field Selection set Argument Variable — Fragment — Alias Directive

TOOLING

Apollo Client urql Relay TanStack Query + graphql-request
gql.tada Houdini GraphiQL / Apollo Sandbox GraphQL LSP (VS Code)

SCHEMA CONCEPTS

Type Field Nullability (🟢) Resolver Schema-first Code-first

* adding enum values is a breaking change

SDL TYPES

Object type Scalar Enum ← * Interface ← Union ← Input type ←

SERVERS BY LANGUAGE

Apollo Server / Yoga (JS/TS) Hot Chocolate (.NET) Strawberry /
Ariadne (Python) gqlgen (Go) graphql-ruby (Ruby) Juniper
(Rust) Lighthouse (PHP/Laravel)

In the AI era, this distinction makes you a better prompter.

■ "Write me a GraphQL query"

and

■ "Write me a GraphQL schema"

are different instructions.

If you conflate them, your agent will too.


```
"You're not learning GraphQL.  
GraphQL is teaching you – about your own API."
```


■ Section 3: Production

■ The questions you'll be prompting in three months.

```
"Why is my GraphQL query slow?  
It only requests five fields."
```



```
"Someone is querying my entire GraphQL schema.  
Types, fields, everything. How do I stop that?"
```



```
"My API got hit with a query that nested fifteen levels deep  
and took the server down."
```



```
"Where does authorization go in GraphQL?  
The framework doesn't seem to have an opinion."
```

The baseline: resolver-level authorization.

The resolver is just a function. Check permissions before returning data.

The better answer: directive-based access control at the schema layer.

```
type Query {  
  adminDashboard: Dashboard @authenticated @requiresScopes(scopes: ["admin"])  
}
```

Authorization logic out of your resolver bodies. In the schema where it's visible.


```
"How do I know which GraphQL operations  
my clients are actually running?"
```


At the very start of this talk – before I introduced myself – I said something about this query.

```
"Notice it has a shape. Different parts of it are going to be  
rendered by different UI components. There's a concept called  
fragments that formalizes exactly that relationship.  
We'll come back to it at the end."
```

We already used fragments once.

`FlightDetails` — to avoid repeating ourselves across the query and the mutation.

That's the DRY version. Useful. But there's a deeper version.


```
"A list of flight cards. Each card has the airline,  
times, maybe a price badge."
```


`FlightList` → `FlightCard` → `PriceBadge`

Each one is a component. Each component has a specific set of fields it needs to render. No more, no less.

The component should own its data requirements.

Not the page. Not the query. The component.

If `PriceBadge` needs `amount` and `currency`, that's `PriceBadge`'s concern — not `FlightCard`'s, not the query author's.

Fragments make that ownership explicit.

■ The naming convention

`ComponentName typeName`

Tells you who owns this fragment and what type it's on.

Your agent will generate anonymous fragments or lazy names.

Tell it the convention upfront.

When a designer changes the `PriceBadge` component, the developer who owns it updates one fragment.

Nothing else breaks.

